

Security Audit

Beamable Network (DePIN)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Beamable Network team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Beamable Network has launched an onchain protocol that serves as a tokenized economy for compute, enabling service provisioning from a distributed network that brings resilience and cost-efficiency to the Cloud Compute model. Built from a successful tech platform that earned trust and success in the Games Industry, the project's goal is to continue scaling the protocol to the benefit of customers and token holders, alike. The protocol touts enterprise-grade infrastructure, sustainability through decentralized operation, and high scalability that competes with the dominant hyperscaler industry model.

Project Name: Beamable Network

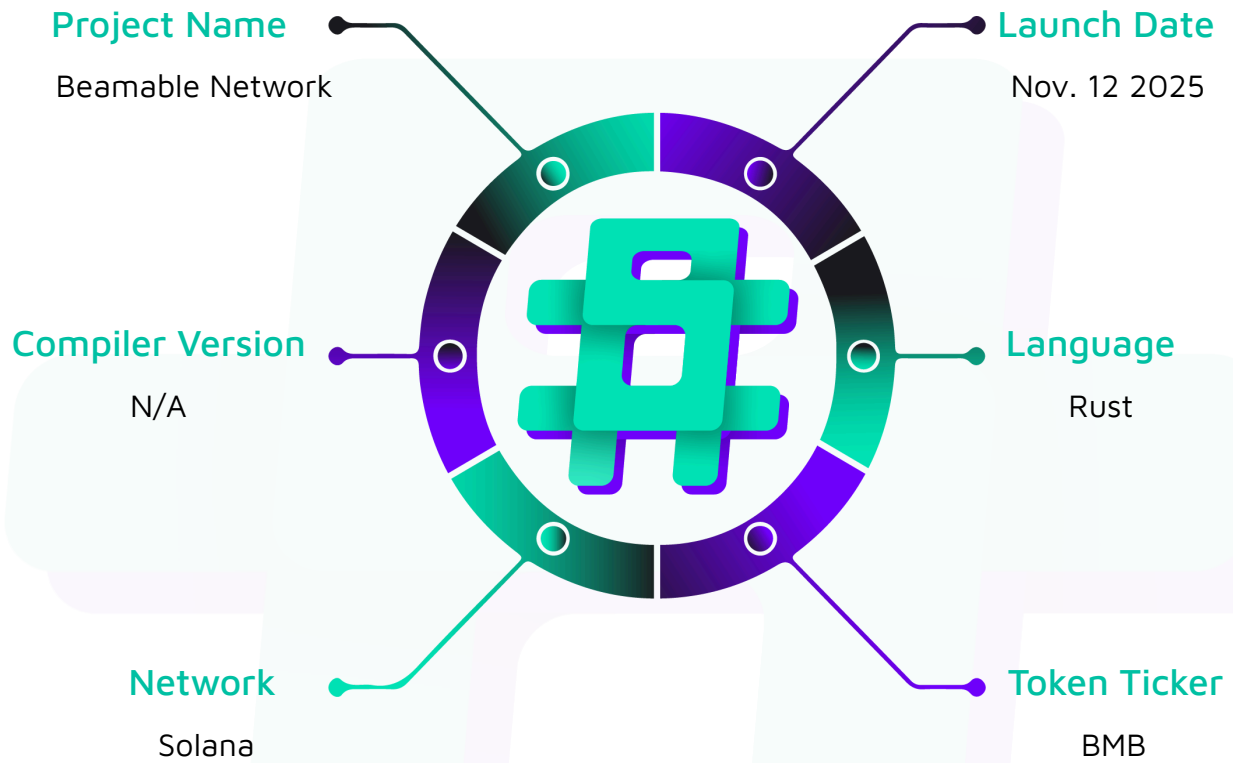
Project Type: DePIN

Compiler Version: N/A

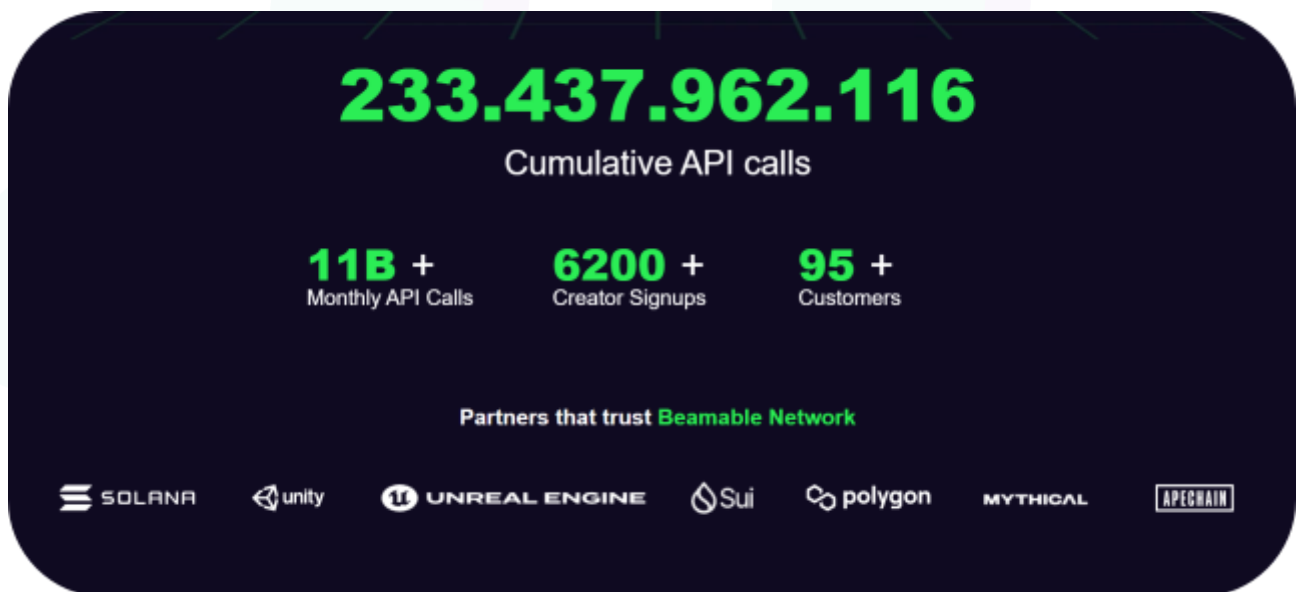
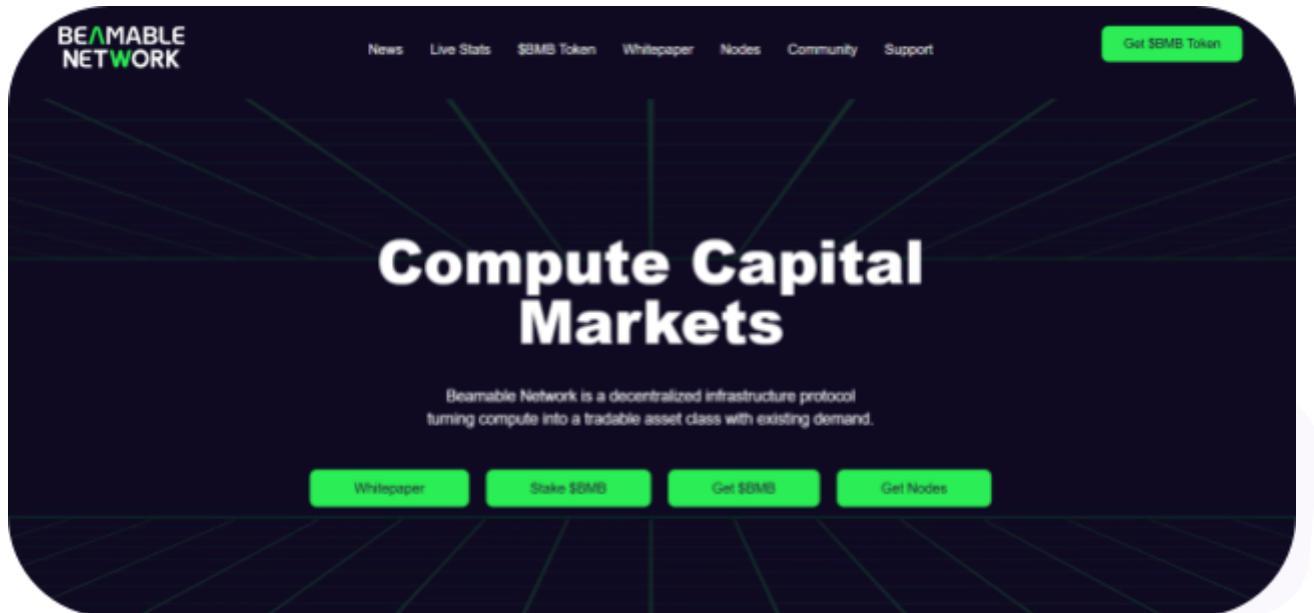
Website: <https://beamable.network>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Beamable Network project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Beamable Network Worker Node Staking Pool Contract
Platform	Solana / Rust
Audit Date	November, 2025
Contract 1	worker_stake/
Audited GitHub Commit Hash	5c31e3de69a028caafad2fc839eb08e4d1d007b5
Fix Review GitHub Commit Hash	2473a5336887375d684cbb102b8d91d07f17d353

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
instructions/admin/ - Initialize_worker_stake_config.rs - Initialize WorkerStakeConfig PDA with worker collection, wallet, and minimum stake requirement. - Update_min_stake_requirement.rs - Update minimum stake requirement.	Contract achieves this functionality.
instructions/user/ - stake.rs - Stake BMB tokens to community vault, initialize pool with inheritance, update user position with time-weighted rewards. - unstake.rs - Opt-out of pool, update opted-out totals in current active pool, prevent re-staking. - claim_rewards.rs - Sequential reward claiming with stake/points ratio calculation, distribute USDC and BMB emissions. - withdraw.rs - Withdraw staked BMB after unstaking, close UserStakePosition PDA, return rent.	Contract achieves this functionality.
instructions/worker/ - worker_stake.rs - Worker stakes BMB to dedicated vault with min stake validation. - worker_unstake.rs - Worker unstakes with global total validation. - set_monthly_pool.rs - Create/update monthly pools, compact created_pools list, validate fee percentages. - update_worker_wallet.rs - Update worker wallet for emission deposits.	Contract achieves this functionality.

instructions/deposits/ - deposit_revenue.rs - Deposit USDC revenue to pools, split between base/addon/worker based on percentages. - deposit_emissions.rs - Deposit BMB emissions with retroactive past-month support, split between base pool and worker.	Contract achieves this functionality.
states/ - worker_stake_config.rs - Top-level config - worker collection, wallet, min stake requirement, total staked, last active pool, created_pools Vec. - monthly_pool.rs - Pool accounting - base/addon metrics with time-weighting, revenue/emission percentages, collected amounts, initialization flag. - user_stake_position.rs - Per-user tracking - staked amount, stake entries with timestamps, opted-out flag, last claimed month.	Contract achieves this functionality.
utils/mod.rs - Pool inheritance logic (stakes and points only), time-weighted calculations, collection validation.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Beamable Network project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Beamable Network project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `claim_rewards` - `created_pools` Compaction Locks Rewards Forever

Description

The `claim_rewards` function checks the `created_pools` Vec to determine if a pool exists, but `set_monthly_pool` compacts this list by removing past months without closing the actual `MonthlyPool` PDAs. Users cannot claim rewards from compacted months even though their funds remain locked on-chain.

Vulnerability Details

When `set_monthly_pool` creates a new pool, it removes past months from the tracking list:

```
config.created_pools.retain(|&m| m >= current_month_period);
```

But `claim_rewards` uses this list as a gatekeeper before checking actual PDA state:

```
if !config.created_pools.contains(&month_period) {  
    user_position.last_claimed_month_period = month_period;  
    msg!("No pool exists for month {}, skipping with 0 rewards", month_period);  
    return Ok(()); // Early return - never checks if PDA actually exists  
}
```

When a user tries to claim from month 5 after it's been removed from `created_pools`, they receive 0 rewards and their `last_claimed_month_period` is set to 5. The sequential claiming validation prevents them from ever claiming month 5 again, permanently locking their rewards in the `MonthlyPool` PDA.

Impact

All users with unclaimed rewards in compacted months permanently lose 100% of their funds.

Recommendation

Remove the `created_pools` gatekeeper check and validate actual PDA state using `monthly_pool_account.data_is_empty()` and `monthly_pool.initialized` instead. Treat `created_pools` as a tracking list only, not the source of truth for pool existence.

Status

Resolved

[H-02] worker_unstake.rs#process_worker_unstake - Workers can't perform a full exit once they staked

Description

Currently, it's not possible for the workers to fully exit the system once they become the workers.

Vulnerability Details

There's a following minimum stake requirement enforced when workers unstake:

```
if new_total_staked < config.min_stake_requirement {  
    msg!(  
        "Error: After unstake, total_staked ({{}}) would be below min_stake_requirement  
({{}})",  
        new_total_staked,  
        config.min_stake_requirement  
    );  
    return Err(ProgramError::InsufficientFunds);  
}
```

This creates two possible issues for the current implementation:

- 1 - if the worker only uses self-staking, then it means that it's not possible to withdraw the full amount for them (including the minimum stake) from the system and it remains stuck
- 2 - except for the self-staking, there's an option to use monthly pools and that creates another issue: when the users unstake, there's no such minimum stake enforced and they can freely withdraw their full position amount. But in the case of a worker, he can't withdraw the amount that falls below the minimum stake -> therefore, in the case when there are monthly pools, the worker can fully withdraw his amount before the other users do - it works because the total stake is the shared value between self-staking and monthly pools - and this will lead to the loss of funds for those users

who staked in the monthly pools as they won't be able to withdraw because of the `config.total_staked` underflow.

Impact

Users lose their funds or they become stuck (in the case of workers).

Recommendation

The fix is non-trivial - consider allowing the workers to withdraw their full amount or change the placement of the minimum stake check so that the other users can't be affected by the workers' withdrawals.

Status

Resolved

Medium

[M-01] `claim_rewards#process_claim_rewards` - Missing Sequential Validation on First Claim Causes Permanent Reward Loss

Description

First-time claimers can skip earlier months and permanently lose earned rewards due to missing validation. The sequential claiming check only enforces order when `last_claimed_month_period > 0`, allowing users to claim any month as their first claim. This validation gap creates a trap where users lose rewards from earlier months permanently.

Vulnerability Details

The function only validates sequential claiming after the first claim has been made:

```
if user_position.last_claimed_month_period > 0 {  
  
  // BUG: When last_claimed_month_period == 0, no validation occurs  
  
  if month_period != user_position.last_claimed_month_period + 1 {  
  
    return Err(ProgramError::InvalidArgument);  
  
  }  
  
}
```

For example: User stakes in months 1-5 earning rewards. On the first claim, they call `claim_rewards(month=5)`. Since `last_claimed_month_period` is 0, this check is skipped. Month 5 is claimed successfully, setting `last_claimed_month_period=5`. Months 1-4 become permanently inaccessible as subsequent claims must follow sequential order.

Impact

First-time claimers permanently lose rewards from earlier months if they claim out of order, with no recovery mechanism available.

Recommendation

It's recommended that the function validates the first claim must be the user's earliest staked month:

```
if user_position.last_claimed_month_period == 0 && user_position.stake_entries.is_empty()
{
    let first_staked_month = user_position.stake_entries[0].month_period;

    if month_period != first_staked_month {
        msg!("Error: First claim must be earliest staked month {}", first_staked_month);
        return Err(ProgramError::InvalidArgument);
    }
}
```

Status

Resolved

[M-02] worker_unstake.rs#process_worker_unstake - Associated token account is not closed and lamports are not refunded if there's a full exit performed by a worker

Description

At the moment, the worker collection update authority pays for the initialization of the worker stake vault ATA. However, if the worker performs a full exit (`vault_account.amount == 0`), then

Vulnerability Details

Consider the following functionality:

```
// Initialize worker stake vault ATA if needed (lazy initialization)

initialize_ata_if_needed(

    collection_authority_account,

    worker_stake_vault_pda,

    bmb_mint_account,

    worker_stake_vault_ata,

    token_program,

    associated_token_program,

    system_program,

)?;
```

As you can see here, we initialize the `worker_stake_vault_ata` that will hold BMB as a worker's stake. The problem is that the account is not closed once the worker unstakes. In the case of the monthly pools, when a user unstakes, the lamports (used for the rent) from the user position PDA are transferred back.

Impact

As ATA is not closed, the lamports stay in the account even though it's not used anymore - the refund is not performed.

Recommendation

Close the account when the amount becomes zero and the worker can't operate no more on the network.

Status

Resolved



Low

[L-01] deposits#deposit_emissions, deposit_revenue - Missing Amount Zero Validation Enables Transaction Spam

Description

The `deposit_emissions` and `deposit_revenue` functions accept `amount` and `total_revenue` parameters without validating against zero. Since these functions only require a signer (no special role checks), anybody can call them with zero amounts to spam transactions. When called with zero amounts, all validations pass and the function executes, enabling transaction spam.

```
// deposit_emissions.rs:32
pub fn process_deposit_emissions<'a>(
    amount: u64, // No validation for amount == 0
) -> ProgramResult {

// deposit_revenue.rs:31
pub fn process_deposit_revenue<'a>(
    total_revenue: u64, // No validation for total_revenue == 0
) -> ProgramResult {
```

Recommendation

Add validation for zero amount check.

```
if amount == 0 {
    return Err(ProgramError::InvalidArgument);
}
```

Status

Resolved

[L-02] deposit_emissions.rs#process_deposit_emissions - Double check of the worker wallet

Description

The worker's wallet is currently double checked:

```
if *worker_wallet_account.key != config.worker_wallet {  
    msg!("Error: Worker wallet account does not match config");  
    return Err(ProgramError::InvalidArgument);  
}
```

And, after that, there's the same check when executing the logic inside of the if-else statement (if the pool exists):

```
if *worker_wallet_account.key != worker_wallet {  
    msg!("Error: Worker wallet account does not match config");  
    return Err(ProgramError::InvalidArgument);  
}
```

Recommendation

Remove the checks in both if and else statements as there's already existing one check before them.

Status

Resolved

[L-03] deposit_emissions.rs#process_deposit_emissions - Unreachable logic when withdrawing from the pool

Description

At the moment, there's the following logic for the last claimable month derivation

```
let last_claimable_month = if user_position.opted_out_at_month_period > 0 {
    user_position.opted_out_at_month_period.saturating_sub(1) // Can only claim up
to month before opted out
} else {
    current_month_period.saturating_sub(1) // Can claim up to previous month (not
current) };;
```

As you can see, if the user has already opted out, we get the last claimable month by subtracting 1 from the opted out month. However, the else-statement in this case can't be invoked at all as before we check for this:

```
// Check if user can withdraw (must have opted out and waited)

// Always require unstake first to properly update last active pool's opted_out
accounting

if user_position.opted_out_at_month_period == 0 {
    msg!("Error: Must call unstake first before withdrawing");
    return Err(ProgramError::InvalidArgument);}
```

So if we didn't opt out, the function can't even be called.

Recommendation

Remove the logic in the else-statement or change the current implementation to account for it.

Status

Resolved

[L-04] stake.rs#process_stake - Excessive drop logic

Description

The drop functionality is unnecessarily used in the `process_stake` function:

```
// Save all updated accounts

let mut position_data = user_position_account.try_borrow_mut_data()?;

write_account_data(
    &mut position_data,
    WorkerStakeAccountType::UserStakePosition,
    &user_position,
)?;

drop(position_data);

let mut pool_data = monthly_pool_account.try_borrow_mut_data()?;

write_account_data(
    &mut pool_data,
    WorkerStakeAccountType::MonthlyPool,
    &monthly_pool,)?;

drop(pool_data);
```

Here, we don't need to drop the data as the borrow is performed on the different accounts - this case is different when we read the data and drop the data afterwards.

Recommendation

Remove the unnecessary logic.

Status

Resolved

QA

[Q-01] worker_stake - Magic Numbers Should Use Named Constants

Description

Magic numbers (86400, 33, 1, 30) are used directly in calculations across many locations without named constants.

- `depin_core/src/utls/bmb.rs`
- `worker_stake/src/instructions/worker/set_monthly_pool.rs`
- `worker_stake/src/utls/mod.rs`

Recommendation

```
pub const SECONDS_PER_DAY: i64 = 86_400;

const COLLECTION_KEY_SIZE: usize = 1;

const PUBKEY_SIZE: usize = 32;

const COLLECTION_HEADER_SIZE: usize = COLLECTION_KEY_SIZE + PUBKEY_SIZE; // 33

const AVERAGE_MONTH_DAYS: u64 = 30;
```

Status

Resolved

[Q-02] worker_stake - Inconsistent Use of PDA Validation Helper

Description

Few functions perform manual PDA validation instead of using the existing `validate_pda_account` helper.

- `worker_stake/src/instructions/admin/initialize_worker_stake_config.rs`
- `worker_stake/src/instructions/worker/set_monthly_pool.rs`
- `worker_stake/src/instructions/worker/worker_stake.rs`
- `worker_stake/src/instructions/worker/worker_unstake.rs`
- `worker_stake/src/instructions/worker/update_worker_wallet.rs`

Recommendation

```
// Manual validation

if *worker_stake_config_account.key != config_pda {

    return Err(ProgramError::InvalidArgument);

}

// Should use helper

validate_pda_account(worker_stake_config_account, &config_pda, "WorkerStakeConfig");
```

Status

Resolved

[Q-03] account#reallocate_account_if_needed - Unchecked Arithmetic in Rent Refund Operations

Description

The `reallocate_account_if_needed` function uses unchecked arithmetic operators (`-=` and `+=`) when transferring lamports during rent refunds. This is inconsistent with the established safe pattern used in `withdraw.rs` which uses `checked_add()` for lamport operations.

Recommendation

Use checked arithmetic consistent with the pattern.

```
let target_starting_lamports = target_account.lamports();

**target_account.try_borrow_mut_lamports()? = target_starting_lamports

    .checked_sub(excess_rent)

    .ok_or(ProgramError::ArithmeticOverflow)?;

let payer_starting_lamports = payer.lamports();

**payer.try_borrow_mut_lamports()? = payer_starting_lamports

    .checked_add(excess_rent)

    .ok_or(ProgramError::ArithmeticOverflow)?;
```

Status

Resolved

[Q-04] worker_stake#initialize_worker_stake_config - Inconsistent Min Stake Validation

Description

The `initialize_worker_stake_config` function accepts `min_stake_requirement` without validating against zero, while `update_min_stake_requirement` enforces `new_min_stake_requirement > 0`. This inconsistency could allow initializing the config with zero minimum stake requirement, which the protocol later disallows for updates.

Recommendation

Add validation before creating the config.

```
if min_stake_requirement == 0 {  
    msg!("Error: min_stake_requirement must be greater than 0");  
    return Err(ProgramError::InvalidArgument);  
}
```

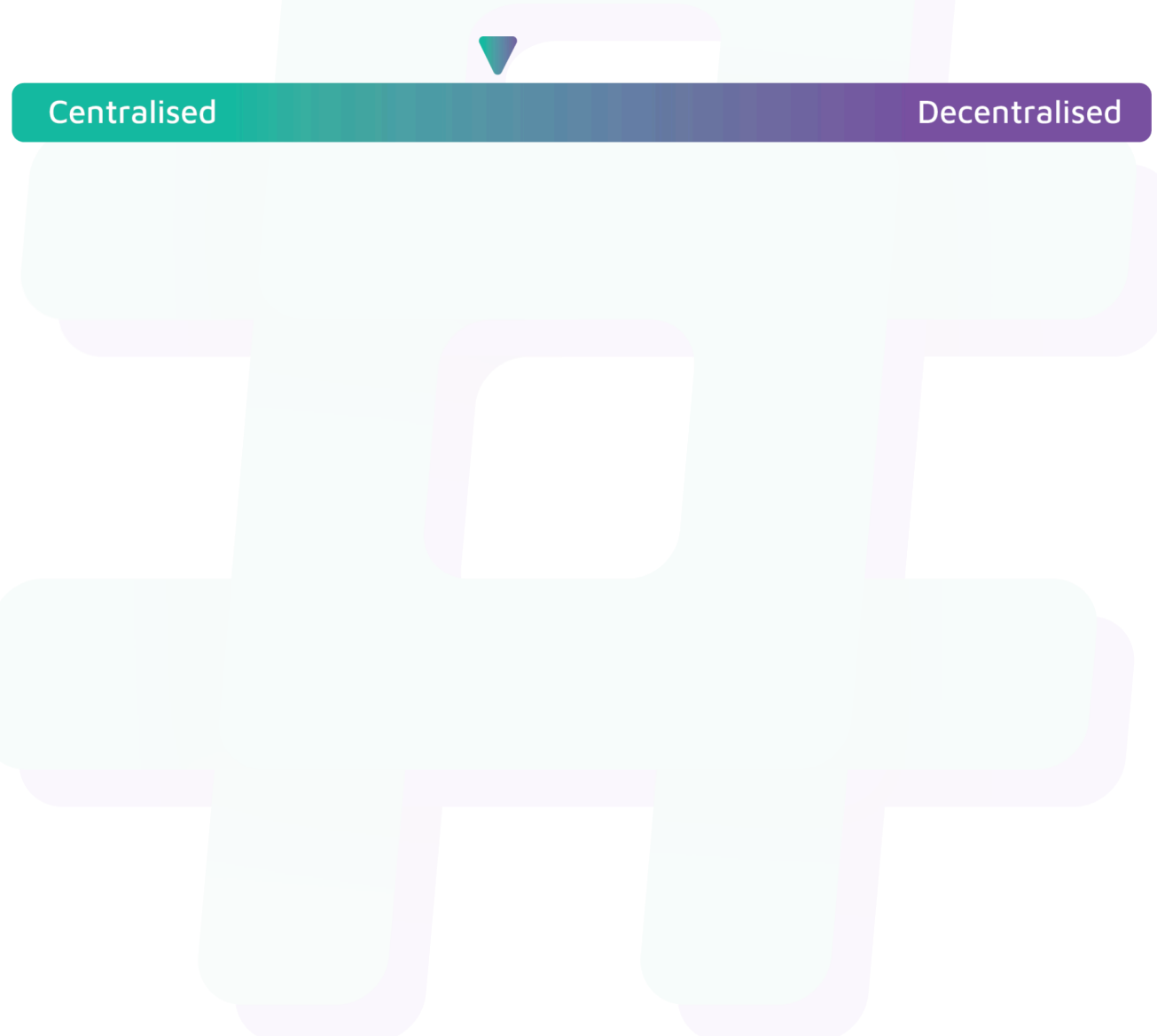
Status

Resolved

Centralisation

The Beamable project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Beamable Network project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.